

```
distance=models.Distance
.COSINE),
)
print(f"Collection '{collection_name}' created with vector
size {vector_size} and
Cosine distance.")
```

- *collection_name*: A unique name for our collection.
- *vector_size*: Obtained directly from *embedding_model.get_sentence_embedding_dimension()*. It's crucial that this matches the output size of your embedding model.
- *models.Distance.COSINE*: Cosine similarity is a common and effective distance metric for text embeddings. It measures the cosine of the angle between two vectors, ranging from -1 (opposite) to 1 (identical).

The *client.collection_exists* and *client.delete_collection* lines are included for convenience, allowing you to run the script multiple times without errors if the collection already exists.

Figure 3 gives a visual representation of the concept of creating a collection, and presentation of *collection_name* and *vector_size*.

You should be able to see the newly created collection in your Qdrant user interface.

Generate embeddings and upsert into Qdrant: Now we'll generate embeddings for each of our documents and then 'upsert' (insert or update) them into our Qdrant collection. Each point in Qdrant consists of a unique ID, the vector itself, and optionally a payload (metadata associated with the vector).

```
points = []
for i, doc in enumerate(documents):
    vector = embedding_model.encode(doc).tolist() # Convert numpy
    array to list
    point_id = str(uuid.uuid4()) # Generate a unique ID for each
    point
    payload = {"text": doc, "index": i} # Store the original text
    and
    an index as metadata
    points.append(
    models.PointStruct(
    id=point_id,
```

```
from qdrant_client import QdrantClient, models
from sentence_transformers import SentenceTransformer
import uuid

# Initialize Qdrant client
# We are connecting to the Qdrant instance running locally on port 6333
client = QdrantClient(host="localhost", port=6333)

print("Qdrant client initialized.")
```

Figure 3: Initialise the Qdrant client

```
vector=vector,
payload=payload,
)
)
client.upsert(
collection_name=collection_name,
wait=True, # Wait for the operation to complete
points=points,
)

print(f"Upserted {len(points)} points into '{collection_
name}'.")
# Get collection info to verify the number of points
collection_info = client.get_collection(collection_name
=collection_name)
print (f"Current number of points in collection: {collection_
info
.points_count}")
```

- *embedding_model.encode(doc).tolist()*: This takes a document, generates its embedding as a NumPy array, and then converts it to a Python list, which Qdrant expects.
- *uuid.uuid4()*: Generates a universally unique identifier (UUID) for each point. This ensures that each vector has a distinct ID in Qdrant.
- *payload*: This is a dictionary where you can store any additional metadata you want to associate with your vector. Here, we're storing the original text and its index. This metadata is extremely powerful for filtering and enriching search results.
- *client.upsert()*: This method sends our points to Qdrant. *wait=True* makes the client wait for the operation to complete before continuing.

This process essentially transforms your raw text data into a searchable, semantic database.

Step 3: Performing a semantic search

Now that your Qdrant collection is populated with vector embeddings, you can perform a semantic search. This is the core of the tutorial's purpose, as it allows you to find relevant information based on the meaning of your query, rather than just matching keywords.

To perform a search, first convert your search query into a vector using the same *SentenceTransformer* model you used for your documents. Qdrant then finds and returns the points (documents) in the collection with the most similar vectors.

Encode the search query: You need to use the exact same model to encode your query as you used for the documents. This ensures that the query vector and